```
stack<int> s;
if(!s.empty())          <-1
{
    int const value=s.top();          <-2
    s.pop();                          <-3
    do_something(value);
}
```

Not only is it safe in single-threaded code, it's expected: calling `top()` on an empty stack is undefined behavior. With a shared `stack` object, *this call sequence is no longer safe*, because there might be a call to `pop()` from another thread that removes the last element in between the call to `empty()` ❶ and the call to `top()` ❷. This is therefore a classic race condition, and the use of a mutex internally to protect the stack contents doesn't prevent it; it's a consequence of the interface.

What's the solution? Well, this problem happens as a consequence of the design of the interface, so the solution is to change the interface. However, that still begs the question: what changes need to be made? In the simplest case, you could just declare that `top()` will throw an exception if there aren't any elements in the stack when it's called. Though this directly addresses this issue, it makes for more cumbersome programming, because now you need to be able to catch an exception, even if the call to `empty()` returned `false`. This essentially makes the call to `empty()` completely redundant.

If you look closely at the previous snippet, there's also potential for another race condition but this time between the call to `top()` ❷ and the call to `pop()` ❸. Consider two threads running the previous snippet of code and both referencing the same `stack` object, `s`. This isn't an unusual situation; when using threads for performance, it's quite common to have several threads running the same code on different data, and a shared `stack` object is ideal for dividing work between them. Suppose that initially the stack has two elements, so you don't have to worry about the race between `empty()` and `top()` on either thread, and consider the potential execution patterns.